

Phase 4: Project Planning Phase

Project Name: **FitSense (FitTrack AI)** — Work Breakdown, Milestone Planning & Risk Strategy

4.1 Planning Phase Scope & Methodology

The project planning phase translates the conceptual goals and design models of FitSense into an actionable development roadmap. Given that strict calendar deadlines are not specified in the project document, planning is governed qualitatively across the five documented development epics. This structure aligns environment setup, database modeling, REST API construction, and Google Gemini AI integration into distinct, verifiable delivery milestones.

4.2 Work Breakdown Structure (WBS) Across Documented Epics

Project Epic	Milestone / Stage	Key Development Tasks	Deliverable
Epic 1	Architecture Formulation	Define layered MVC architecture; create Entity Relationship (ER) diagrams; define User, Workout, Category, and AI interaction flows.	System architecture diagram & ER specifications.
Epic 2	Setup & Environment	Initialize root folder AI-FitTrack-API and server sub-directory; execute <code>npm init -y</code> ; install runtime and dev dependencies; build <code>.env</code> file.	Configured <code>package.json</code> & environment variables.
Epic 3	Backend Module Layout	Partition source code into modular layers (<code>config</code> , <code>controllers</code> , <code>middleware</code> , <code>models</code> , <code>routes</code> , <code>services</code>).	Clean directory scaffold inside <code>server/src</code> .
Epic 4	Database & Schemas	Configure Mongoose connection handler (<code>db.js</code>); write <code>User.js</code> with pre-save bcrypt hashing; write <code>Workout.js</code> with category enums and boundaries.	Connected MongoDB database & active schemas.
Epic 5	Execution & Verification	Start Express server daemon; assemble Thunder Client / Postman collections; verify Auth, Workout CRUD, and Gemini AI endpoints.	Verified REST endpoints & execution test log.

4.3 Module-Wise Development Planning

Phase A: Foundation & Security Layer

- **Database Connection:** Implement asynchronous connection module with error trap and console logging.
- **User Entity:** Enforce unique regex email matching and automated timestamp tracking.
- **Auth Middleware:** Build stateless JWT decoder extracting bearer tokens from authorization headers.
- **Controller Logic:** Code user registration, password verification, and profile retrieval logic.

Phase B: Workouts & Intelligence Layer

- **Workout Entity:** Establish 1:N relationship pointing to `User_id`; bind category validation.
- **CRUD Pipeline:** Implement POST, GET, PUT, and DELETE operations restricted to record owners.
- **Search Engine:** Write query filter handlers matching `workoutName`, `category`, and `workoutDate`.
- **Gemini AI Service:** Integrate Google Gemini API client; construct recommendation and insight prompt models.

4.4 Project Timeline & Scheduling Baseline

Calendar Schedule / Dates: *Not specified in the project document.* Progress is structured sequentially through qualitative

4.5 Team Roles & Responsibilities

Identified Engineering Role Assigned Engineer / Developer: Sathya Primary Role: Full-Stack Backend & API Systems Engineer Responsible for end-to-end design, modular scaffolding, schema validation, route shielding, Gemini AI prompt construction, and Thunder Client integration testing.	User Role & Permission Boundary Role: Authenticated User Permissions: Full self-service registration, profile inspection, workout creation, reading, updating, deletion, searching, and AI generation. Additional role tiers (such as Administrator): <i>Not specified in the project document.</i>
--	---

4.6 Resources Required for Execution

Core Tooling & Runtime - Node.js (v16+) runtime - npm (v8+) package manager - VS Code Development IDE - Thunder Client / Postman	Software Libraries - Express.js (REST gateway) - Mongoose & MongoDB daemon - bcryptjs (cryptography) - jsonwebtoken & dotenv	Hardware & Cloud - Intel i5 / AMD Ryzen 5 CPU 8 GB - 16 GB RAM 1 GB disk workspace - Google Gemini API Key
---	---	---

4.7 Risk Identification & Mitigation Strategy

Risk Factor Identified	Severity	Operational Impact	Documented Mitigation Protocol
Unprotected Route Breach	HIGH	Unauthorized access to personal exercise logs and user profiles.	JWT authorization middleware intercepts all private routes, enforcing valid token signatures.
Credential Exposure	HIGH	Plaintext password leakage during database compromise.	Automatic pre-save hook running bcryptjs with 10 salt rounds before saving to MongoDB.
External AI Latency / Down	MEDIUM	Gemini API outages blocking client request threads.	Decoupled service layer architecture and centralized error middleware catching API faults gracefully.
Invalid Data Ingestion	MEDIUM	Database corruption with negative durations or unsupported workouts.	Mongoose schema boundary validations (duration >= 1, calories >= 0, strict category enums).

4.8 Expected Project Deliverables Executable Backend Codebase: Organized into modular folders inside <code>server/src</code>. Database Schemas: Fully configured User and Workout Mongoose models with validation rules. REST API Suite: Nine tested endpoints covering Auth, CRUD Workout Management, and AI services. Environment Configuration: Sample <code>.env</code> file securing database URLs and Gemini API keys. Verification Artifacts: Postman/Thunder Client collection files and recorded video walkthroughs.
